

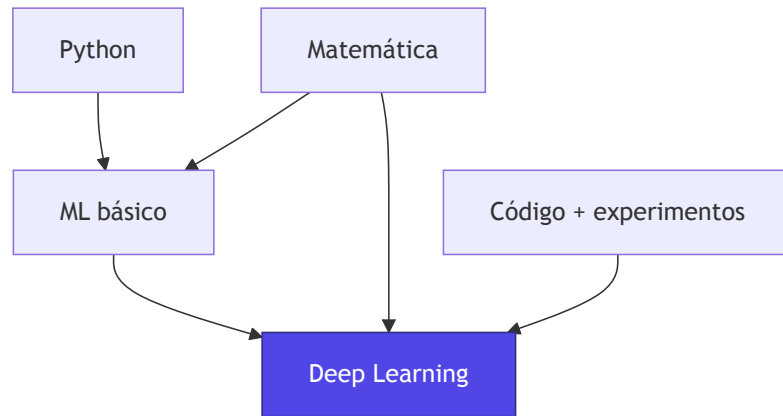
Aula 1

Introdução a Redes Neurais e *Deep Learning*

Tópicos Avançados em Inteligência Artificial · UFABC

Pré-requisitos

- Familiaridade com **Python** em nível intermediário
- Conhecimento de **conceitos fundamentais de Machine Learning**:
 - treino / validação / teste
 - *overfitting* e *underfitting*
 - regularização
- Boa intuição com **álgebra linear e cálculo** (vetores, matrizes, derivadas parciais)
- Disposição para **escrever código** e experimentar



Filosofia do curso

Conceitos antes da matemática · Mão na massa antes da teoria

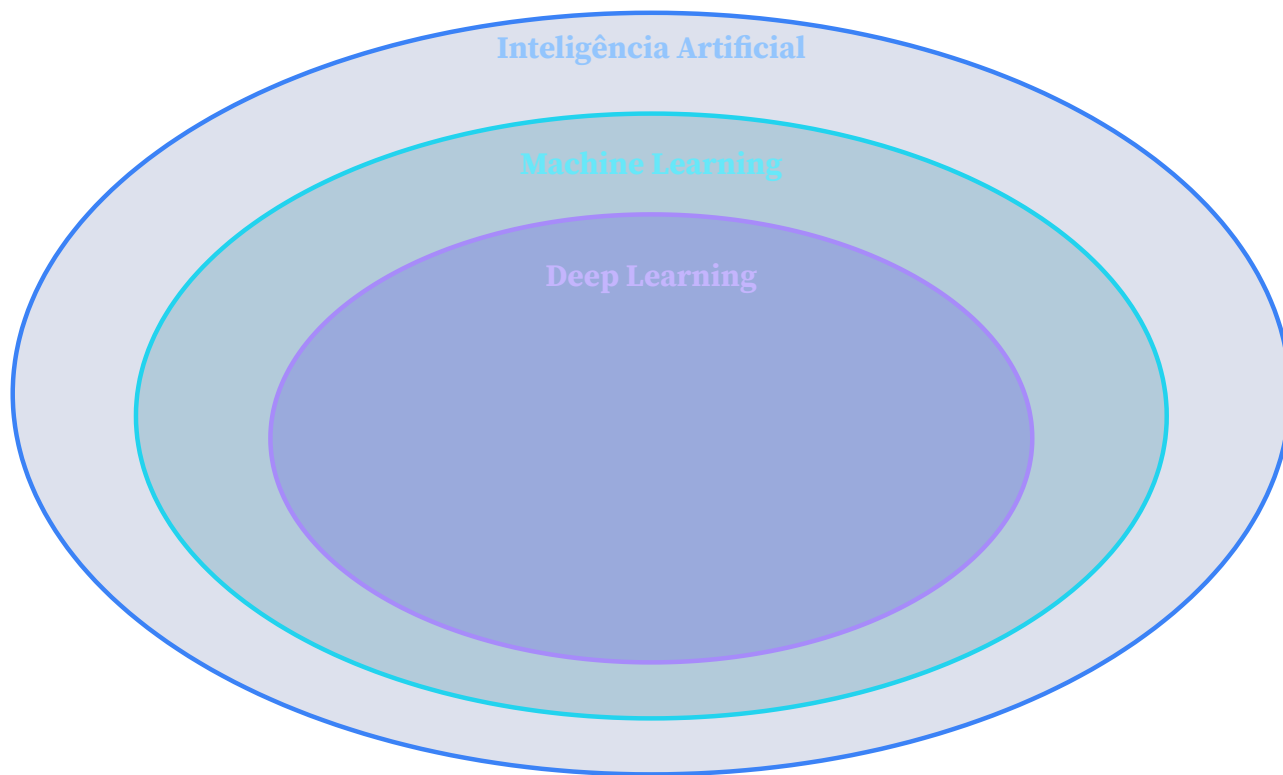
Como vamos abordar o conteúdo

- **Foco nas ideias-chave** que sustentam o *Deep Learning*
- A matemática aparece **quando ajuda**, não como obstáculo
- Aprender DL é como **aprender a nadar**: não dá só assistindo
 - Vamos escrever, treinar e depurar modelos reais
- O objetivo não é formar engenheiros de ML, mas dar autonomia para que vocês construam um **modelo V1.0** sem depender de terceiros

Se você procura uma abordagem fortemente teórica/matemática, há outras disciplinas mais adequadas.

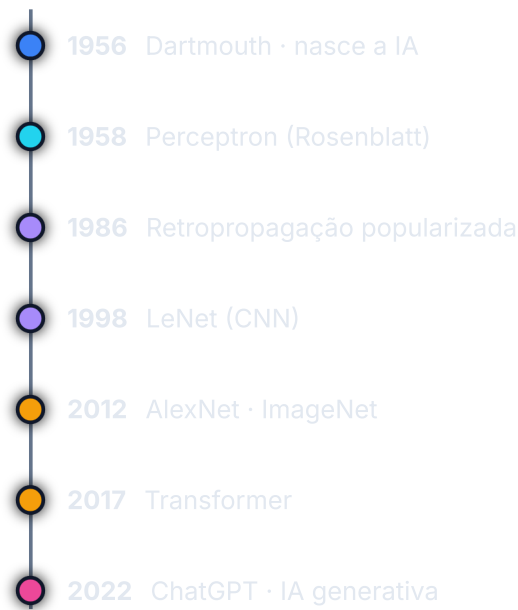
IA, ML, DL e IA Generativa

Antes de mergulhar nas redes neurais, vamos entender as relações entre essas ideias.



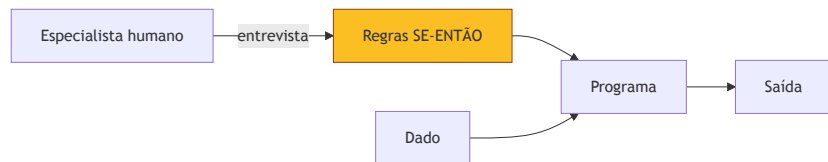
A inteligência artificial nasceu em 1956

- O termo "**Inteligência Artificial**" foi cunhado em uma oficina histórica realizada na **Dartmouth College** (verão de 1956)
- Reunia nomes como **John McCarthy, Marvin Minsky, Claude Shannon, Allen Newell, Herbert Simon**
- Definia uma agenda otimista: máquinas que aprendessem, raciocinassem e usassem linguagem
- Desde então, a IA passou por vários **invernos e renascimentos**



A abordagem clássica de IA

- **Meta:** dar ao computador a capacidade de fazer tarefas que só humanos faziam bem
- **Estratégia clássica:** perguntar a *especialistas humanos* como eles fazem, transcrever em **regras** SE...ENTÃO , programá-las explicitamente
- Funcionou em **alguns domínios bem delimitados** (sistemas especialistas, xadrez baseado em regras)



Por que isso é tão difícil?

"Sabemos mais do que conseguimos contar." — Paradoxo de Polanyi

- Reconhecemos um rosto, andamos de bicicleta, identificamos sarcasmo — mas é **muito difícil escrever as regras** que descrevem como fazemos isso
- Regras explícitas não cobrem **casos extremos** nem generalizam para situações novas
- Resultado: a IA simbólica esbarrou na **complexidade do mundo real**

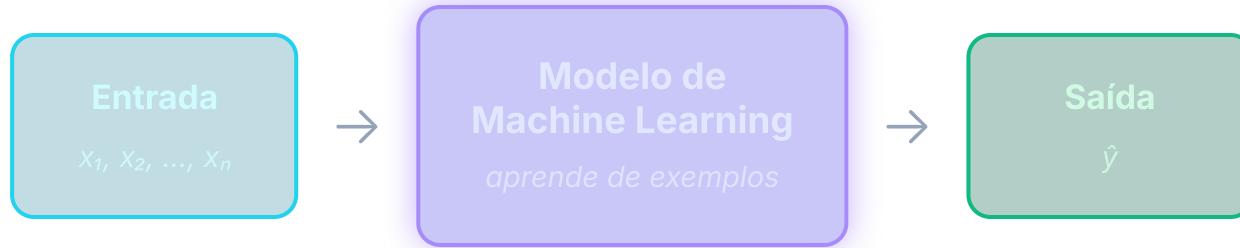
Mudança de paradigma

Em vez de **dizer** ao computador o que fazer...

...**mostre** a ele *muitos exemplos* de entrada e saída, e deixe que **técnicas estatísticas** aprendam a relação.

Isto é Machine Learning

Machine Learning, em uma figura



Algoritmos clássicos: regressão linear, regressão logística, árvores, *random forests*, *gradient boosting*, SVMs, redes neurais rasas...

ML brilha com dados estruturados

- Dados estruturados = aqueles que cabem **naturalmente em uma planilha**
- Cada coluna é uma *feature* numérica/categórica significativa
- ML clássico funciona **muito bem** nesses casos:
 - *score* de crédito
 - previsão de demanda
 - detecção de fraude
 - diagnóstico baseado em exames laboratoriais

idade	renda	tem_imovel	inadimplente
32	5.4k	sim	0
47	3.1k	não	1
25	2.8k	não	0
51	9.2k	sim	0

Exemplo ilustrativo (sintético)

Mas e os dados não estruturados?



Imagens

pixels RGB sem significado isolado



Texto

sequências de caracteres/tokens



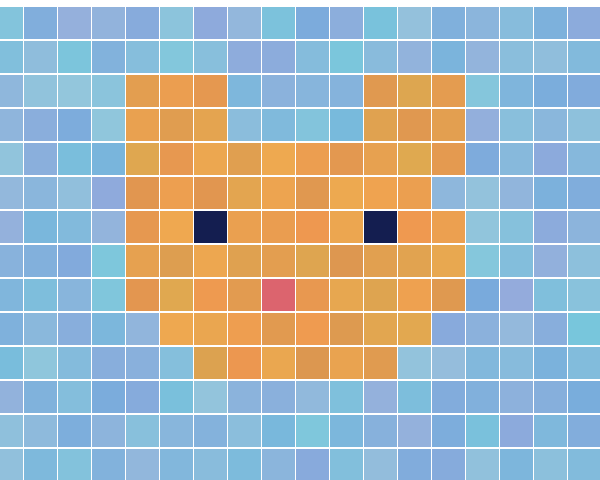
Áudio

amostras temporais brutas

A **forma bruta** desses dados não tem significado intrínseco para os algoritmos clássicos. Um pixel (R=128, G=64, B=200) não diz nada sozinho sobre haver um gato na imagem.

A dificuldade dos dados não estruturados

"O que vemos"



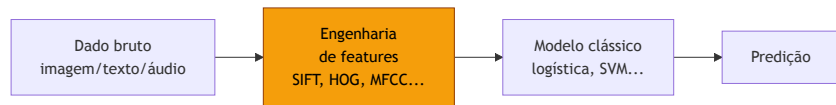
"O que o computador vê"

(131, 196, 220)	(129, 174, 220)	(149, 176, 220)	(146, 179, 220)	(135, 171, 220)	(140, 196, 220)
(129, 191, 220)	(143, 189, 220)	(124, 197, 220)	(130, 178, 220)	(134, 190, 220)	(131, 199, 220)
(143, 183, 220)	(145, 195, 220)	(147, 198, 220)	(139, 196, 220)	(227, 158, 80)	(235, 158, 80)
(143, 181, 220)	(138, 174, 220)	(125, 172, 220)	(144, 198, 220)	(233, 161, 80)	(224, 157, 80)
(145, 196, 220)	(138, 175, 220)	(122, 190, 220)	(121, 181, 220)	(223, 167, 80)	(231, 152, 80)
(147, 184, 220)	(138, 182, 220)	(145, 191, 220)	(143, 170, 220)	(225, 150, 80)	(237, 159, 80)
(148, 177, 220)	(122, 183, 220)	(130, 186, 220)	(147, 180, 220)	(230, 152, 80)	(239, 168, 80)
(136, 178, 220)	(130, 176, 220)	(130, 170, 220)	(126, 199, 220)	(230, 161, 80)	(221, 158, 80)

Um classificador *não* vê o gato — vê uma matriz de números.

A solução pré-DL: engenharia de atributos

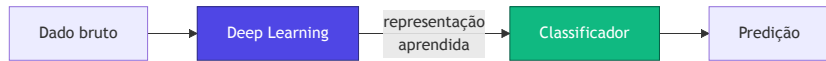
- Especialistas projetavam **representações artesanais** dos dados:
 - **SIFT, HOG, SURF** para imagens
 - **MFCC** para áudio
 - **TF-IDF, *bag-of-words*** para texto
- A representação extraída era então alimentada em um modelo **clássico** (geralmente regressão logística!)



Isso exigia **muito esforço humano** — um *gargalo* que limitou drasticamente o alcance de ML em dados ricos como imagens, voz e linguagem natural.

O que **DL** faz que ML clássico não fazia?

- **Extraí automaticamente** representações úteis dos dados não estruturados
- Essas representações podem alimentar até modelos triviais (uma regressão logística no topo já entrega resultados impressionantes)
- Resolve o **gargalo humano** que limitava ML em imagens, texto e áudio



A engenharia de features deixa de ser *humana* e passa a ser *aprendida*.

Por que aconteceu **agora**?



Algoritmos

ReLU, dropout, batch norm, arquiteturas convolucionais e transformers, otimizadores (Adam, etc.)



Dados

Digitalização de tudo: fotos, vídeos, redes sociais, sensores, logs.
ImageNet, Common Crawl...



Computação

GPUs, depois TPUs. Treinos paralelos massivos em precisão reduzida.

...aplicados a uma ideia **antiga**: as **redes neurais artificiais**.

Aplicação imediata: **percepção**

Cada **sensor** pode ganhar a capacidade de detectar, reconhecer e classificar o que está percebendo.
Acoplar DL a câmeras, microfones e sensores cria produtos qualitativamente diferentes.



Deteção de objetos



Diagnóstico por imagem



Direção autônoma



Inspeção industrial



Reconhecimento de fala



Análise de microscopia



Sensoriamento remoto



Bio-acústica

E quanto à **saída**?

DL clássico previa **saídas estruturadas** com facilidade (um número, um rótulo, um vetor de probabilidades).

Mas **gerar** texto, imagens, áudio, código? Por muito tempo, esse era um **território difícil**.

Saídas que ML/DL conseguia prever bem

- **Um número**

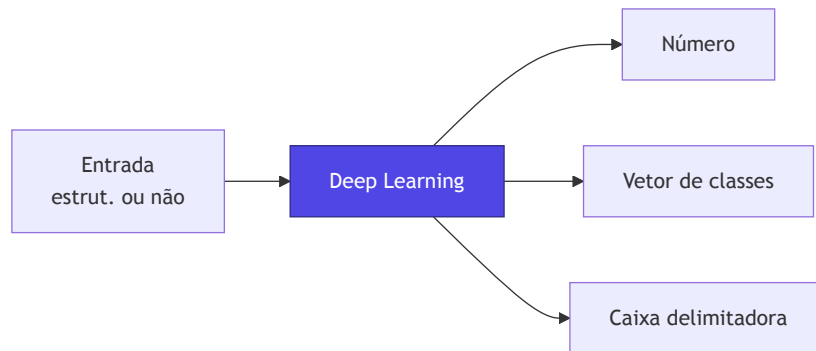
- probabilidade de inadimplência
- demanda da próxima semana

- **Poucos números**

- distribuição sobre 1 000 classes do ImageNet
- coordenadas de um *bounding box*

- **Um rótulo discreto**

- sentimento (positivo/negativo)
- categoria de produto

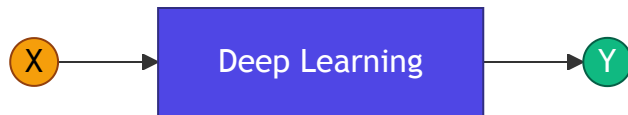


A "matriz" $X \rightarrow Y$ da IA Generativa

Tanto X quanto Y podem ser texto, imagem, áudio, vídeo, código... e até **multimodais**.

$X \setminus Y$		 Texto	 Imagem	 Áudio	 Código
 Texto		tradução, resumo, chat	geração de imagens	TTS, música	geração de código
 Imagem		legendas, OCR	estilização, edição	sonificação	sketch → app
 Áudio		transcrição	visualização	remoção de ruído	—
 Código		documentação	diagramas	síntese sonora	tradução de linguagens

Resumo: $X \rightarrow Y$



Toda a empolgação atual com IA é, em essência, o sucesso do **Deep Learning** (e da IA Generativa que veio sobre ele).

O que é uma rede neural?

Vamos começar do começo.

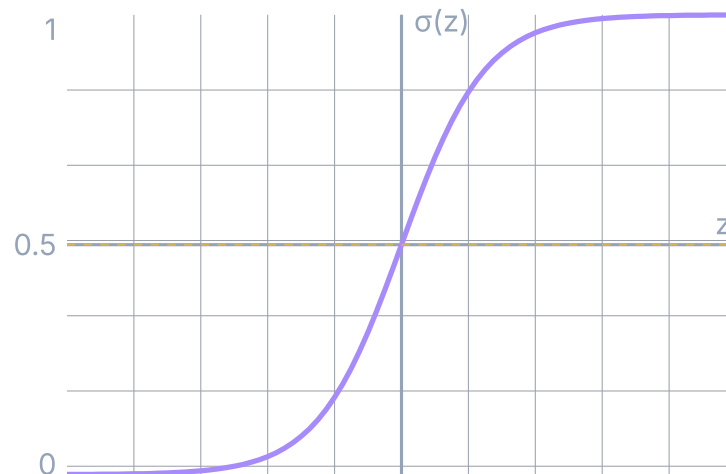
Voltando à regressão logística

A regressão logística mapeia um vetor de entradas em uma **probabilidade**:

$$p(y = 1 \mid \mathbf{x}) = \sigma\left(b + \sum_{i=1}^n w_i x_i\right)$$

onde $\sigma(z) = \frac{1}{1 + e^{-z}}$ é a função **sigmoide**.

Vamos olhar para essa expressão como uma **rede de operações matemáticas**.



$$\sigma(z) = 1 / (1 + e^{-z}) \text{ — saída em } (0, 1)$$

Exemplo: prever quem é chamado para entrevista

- **Entradas:**

- GPA (coeficiente de rendimento)
- Anos de experiência

- **Saída:** probabilidade de ser chamado para a entrevista (1 = chamado, 0 = não chamado)

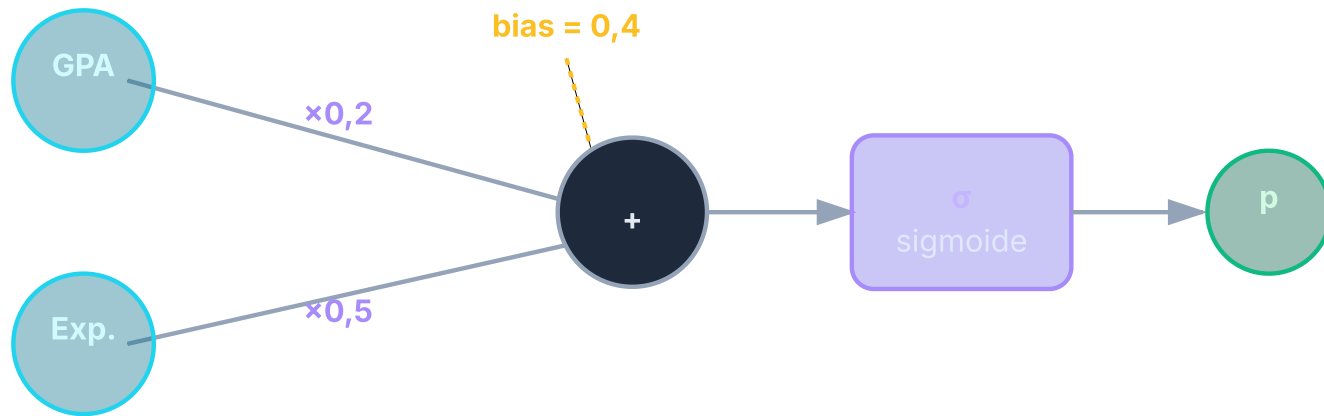
- **Modelo:** regressão logística treinada nos dados históricos

#	GPA	Exp.	Chamado
1	3.27	1.93	0
2	3.37	0.07	0
3	3.57	1.91	0
4	3.91	4.35	0
6	3.90	2.41	1
7	3.94	3.00	1
15	3.77	2.06	1

Suponha que treinamos e obtivemos:

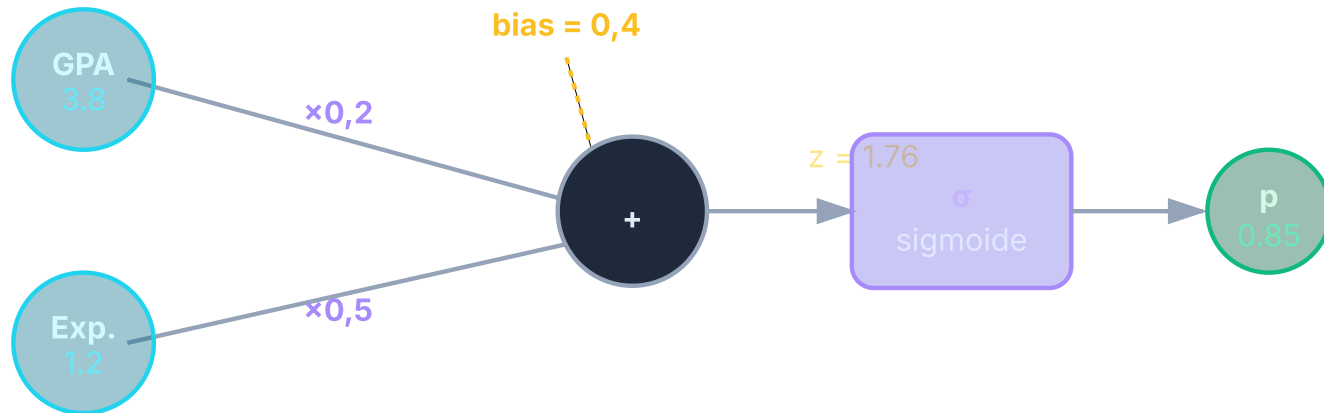
$$p(\text{chamado}) = \sigma(0,4 + 0,2 \cdot \text{GPA} + 0,5 \cdot \text{Exp})$$

Reescrevendo como uma **rede** com fluxo da esquerda para a direita:



Predizendo com a "rede"

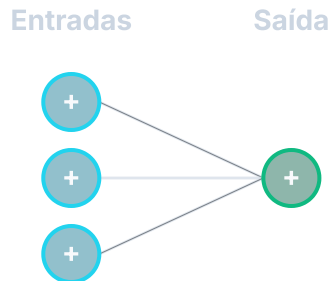
Considere um candidato com **GPA = 3,8** e **1,2 anos** de experiência.



$$\sigma(0,4 + 0,2 \cdot 3,8 + 0,5 \cdot 1,2) = \sigma(1,76) \approx 0,85$$

A probabilidade estimada de chamada para entrevista é cerca de **85 %**.

Vocabulário básico



Pesos — multiplicadores nas conexões entre neurônios

Neurônio — unidade que faz $\sum + \text{bias}$ e aplica ativação

Bias — termo aditivo (intercepto) em cada neurônio

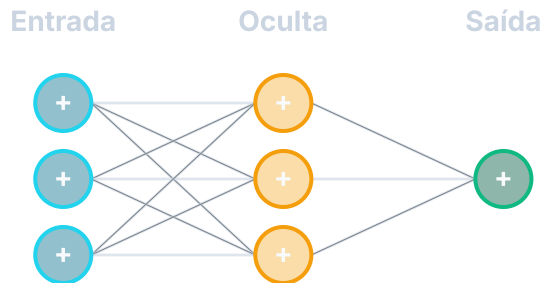
Camada — pilha vertical de neurônios

Por que a "lente de rede" é útil?

- Ela nos permite **transformar** os dados de entrada antes da decisão final
- Em vez de alimentar a regressão logística com os dados crus, podemos primeiro **aprender uma melhor representação** deles
- Esse é o coração do *Deep Learning*: empilhar transformações aprendidas

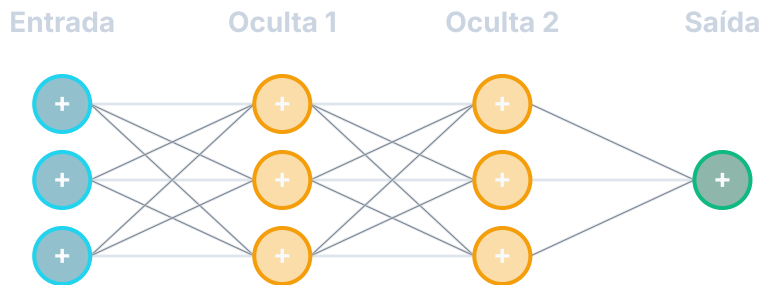
Empilhando funções lineares

Antes da decisão final, **insira** uma camada de funções lineares que combine as entradas:



Cada nó da camada intermediária é $z_j = b_j + \sum_i w_{ji}x_i$ — uma **transformação linear** das entradas.

Empilhando ainda mais



Podemos empilhar **quantas camadas** quisermos. No final, alimentamos a regressão logística (sigmoide) com o vetor transformado.

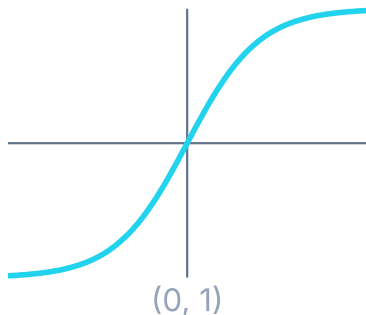
⚠ Mas só com camadas *lineares* a composição segue sendo linear — precisamos de algo a mais.

Funções de ativação não lineares

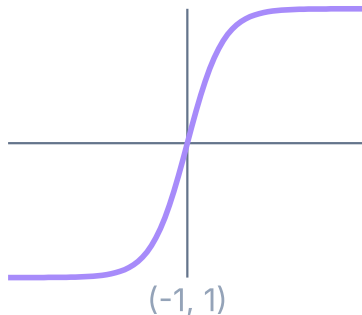
Para que empilhar camadas faça diferença, cada neurônio aplica uma **função de ativação não linear** depois da combinação linear:

$$a_j = \phi\left(b_j + \sum_i w_{ji} x_i\right)$$

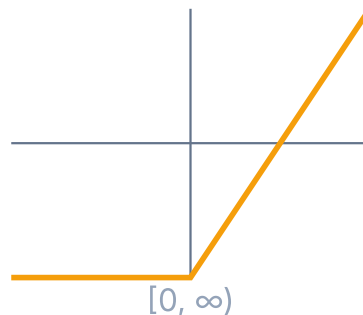
Sigmoide



Tanh



ReLU



Funções de ativação comuns

Sigmoide

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Saída em (0,1). Usada na **camada de saída** para classificação binária. Sofre de *vanishing gradients* em camadas profundas.

Tanh

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Saída em (-1,1), centrada em zero. Mais bem comportada que sigmoide, mas ainda satura.

ReLU

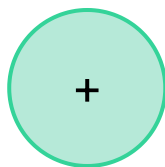
$$\text{ReLU}(z) = \max(0, z)$$

Padrão moderno em camadas ocultas. Barata, evita saturação para $z > 0$, permite redes muito mais profundas.

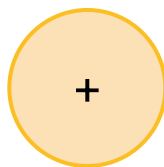
Outras: Leaky ReLU, ELU, GELU, Swish, Softmax (na saída multi-classe)...

Notação visual

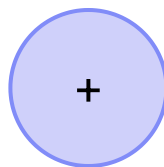
A partir daqui, vamos abreviar cada neurônio com um **círculo**, indicando a ativação por cor/rótulo:



Linear

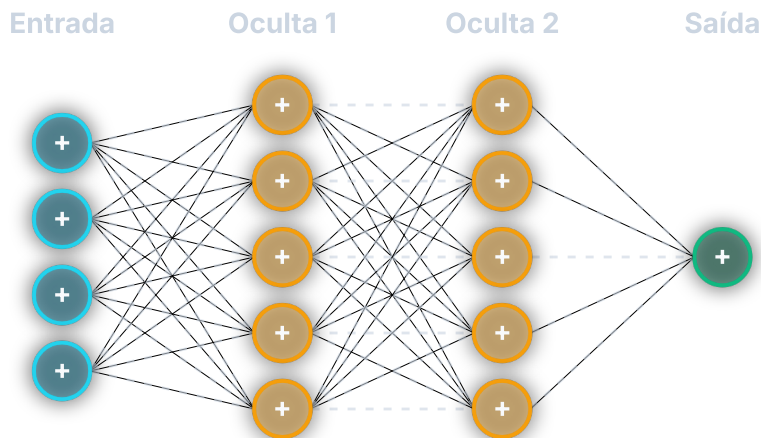


ReLU



Sigmoide

Anatomia de uma DNN



Camada de entrada: as variáveis $x_1, \dots, x_k$

Camada de saída: produz a predição

Camadas ocultas: transformam a representação

Conexões densas: cada neurônio se liga a todos da próxima camada

Quando todas as camadas são densas, dizemos que a rede é *fully connected*.

Projetando uma DNN: as escolhas

Quando você desenha uma rede neural feedforward (*vanilla*), você decide:

Quantas camadas ocultas?

profundidade

Quantos neurônios em cada camada?

largura

Qual ativação nas ocultas?

tipicamente ReLU/GELU

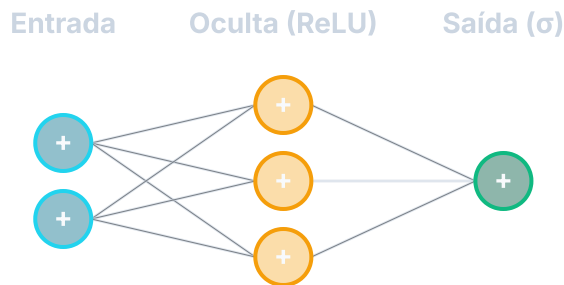
Qual ativação na saída?

depende da tarefa

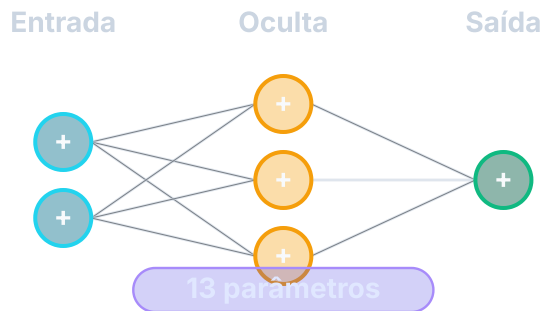
A escolha da ativação de saída é guiada pela natureza de y : linear (regressão), sigmoide (classificação binária), softmax (multiclasse).

Aplicando ao classificador de entrevistas

- **Entradas:** 2 variáveis (GPA, experiência)
- **Saída:** probabilidade $\in (0, 1)$
- **Decisões de projeto:**
 - 1 camada oculta com **3 neurônios** ReLU
 - **Sigmoide** na saída (probabilidade)

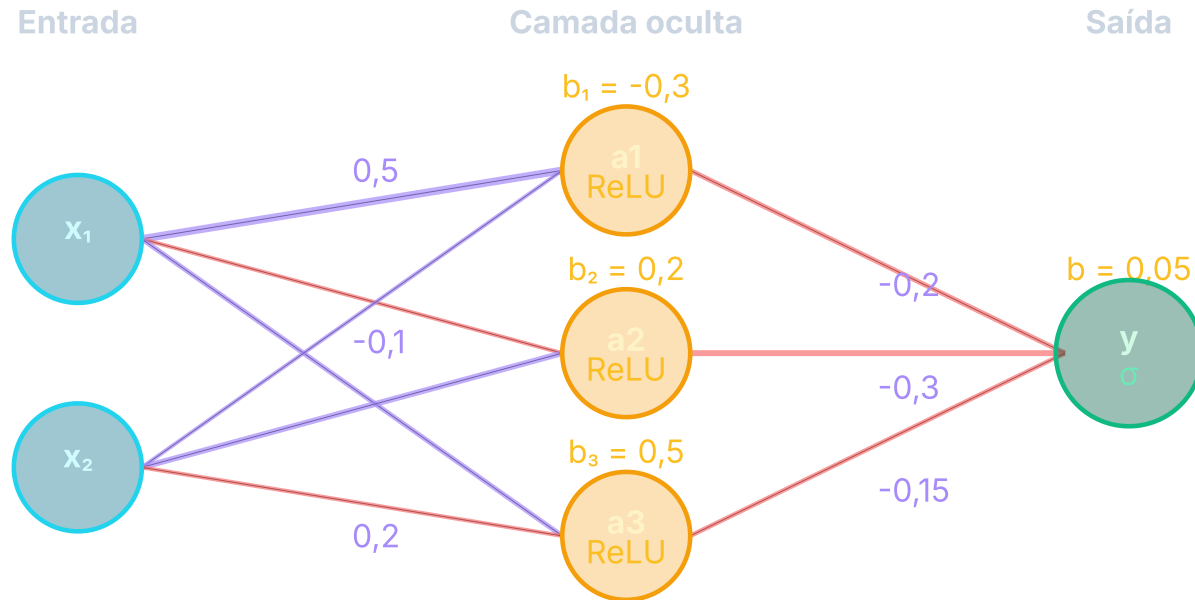


Quantos parâmetros tem essa rede?



- Da entrada para a oculta: $2 \times 3 = 6$ pesos + 3 biases = 9
- Da oculta para a saída: $3 \times 1 = 3$ pesos + 1 bias = 4
- **Total:** 13 parâmetros

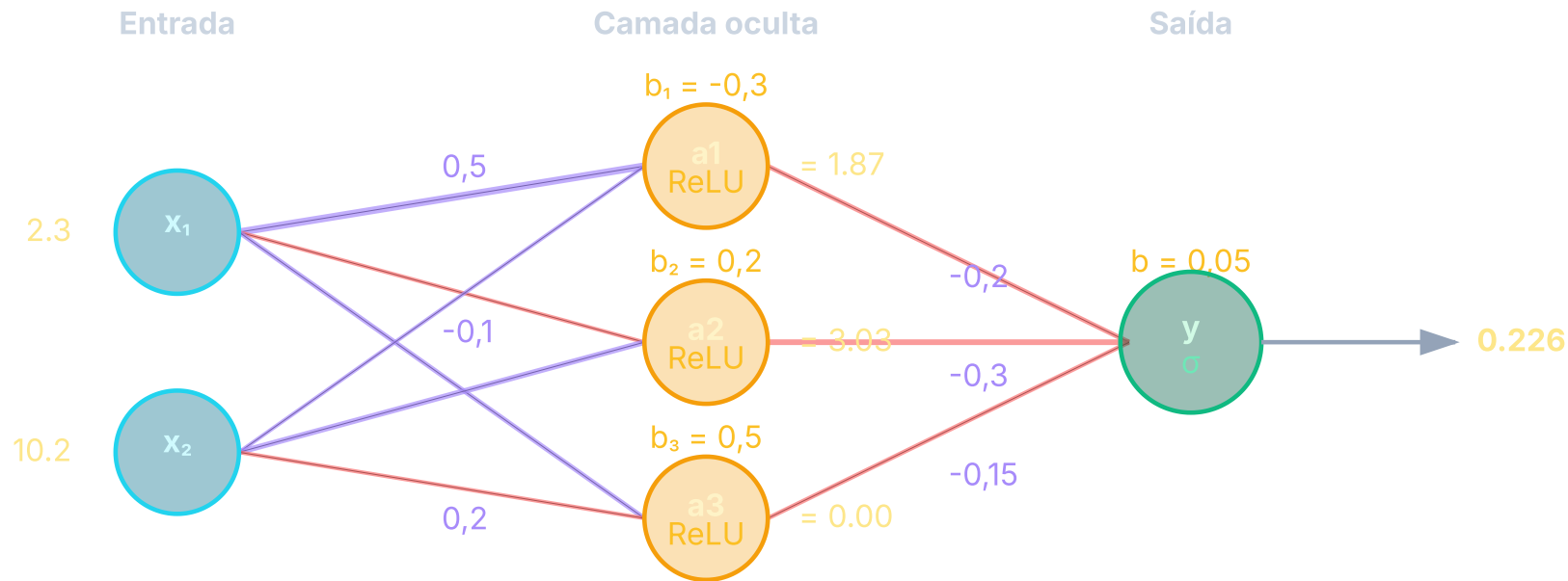
Pesos treinados (suposição)



Como esses pesos são *encontrados* a partir dos dados é o tema da **próxima aula** (treinamento, perda, retropropagação).

Forward pass — calculando uma predição

Para um candidato com $x_1 = 2,3$ (GPA) e $x_2 = 10,2$ (experiência*):



Camada oculta: contas explícitas

$$a_1 = \text{ReLU}(-0,3 + 0,5 \cdot 2,3 + 0,1 \cdot 10,2) = \max(0; 1,87) = 1,87$$

$$a_2 = \text{ReLU}(0,2 - 0,1 \cdot 2,3 + 0,3 \cdot 10,2) = \max(0; 3,03) = 3,03$$

$$a_3 = \text{ReLU}(0,5 + 0,2 \cdot 2,3 - 0,1 \cdot 10,2) = \max(0; -0,06) = 0$$

E a saída:

$$y = \sigma(0,05 + (-0,2) \cdot 1,87 + (-0,3) \cdot 3,03 + (-0,15) \cdot 0) \approx \sigma(-1,23) \approx 0,226$$

Para este candidato, a rede estima $\approx 22,6$ % de chance de chamada.

A rede como uma função

A predição da rede pode ser escrita como **uma única função** das entradas:

$$\hat{y}(\mathbf{x}) = \sigma\left(\mathbf{w}^{(2)\top} \text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + b^{(2)}\right)$$

Regressão logística pura:

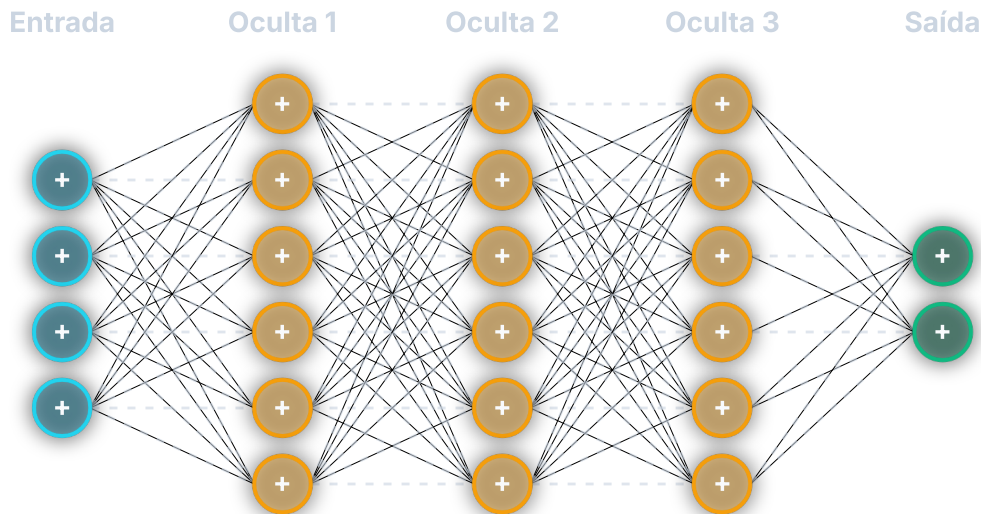
$$\hat{y} = \sigma(0,4 + 0,2x_1 + 0,5x_2)$$

linear nos dados antes da sigmoide.

Rede com 1 camada oculta:

uma composição não linear muito mais rica — capaz de representar relações complexas entre $\mathbf{x}$ e y .

Resumo: uma DNN



- **Feedforward** (vanilla): dados fluem da esquerda para a direita
- **Arquitetura** = camadas + ativações + conexões
- **Profundidade** vem de muitas camadas ocultas
- *Deep Learning* é, em essência, **redes neurais com muitas camadas**
- Treinamento, retropropagação e otimização — próxima aula

Recapitulando

Hierarquia de ideias

IA $\supset$ ML $\supset$ DL $\supset$ IA Generativa

Por que DL é especial

aprende representações de dados não estruturados

A combinação que destravou

algoritmos + dados + GPUs

Rede neural $\approx$ regressão

logística empilhada com não linearidades

Componentes

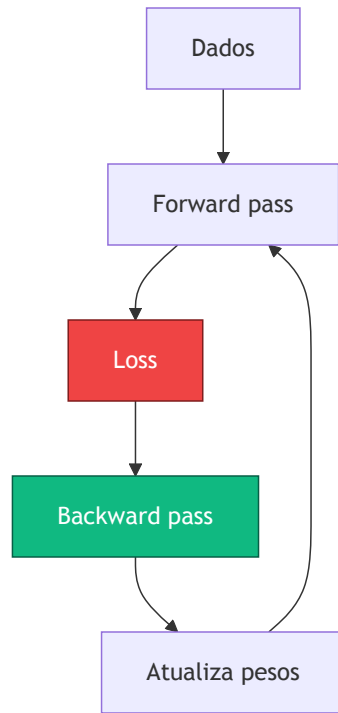
camadas, neurônios, pesos, biases, ativações

Forward pass

composição linear $\rightarrow$ não linearidade $\rightarrow$ repetir

Próxima aula

- Como **treinar** uma rede neural?
- O que é uma **função de perda**?
- **Gradiente descendente** e suas variantes
- **Retropropagação** (intuição + um exemplo numérico)



Obrigado! Perguntas?

Adaptado livremente de *15.773 Hands-on Deep Learning* (MIT OpenCourseWare, 2024) — material original em inglês de Vivek Farias e Rama Ramakrishnan, distribuído sob os termos do MIT OCW.

Para mais informações: <https://ocw.mit.edu/terms>